2025-2026

Welcome to the 16th Lab session of CO3519 module (Semester 02)

For Two Hours Lab Session:

#	Task	Time (m)	Guidance
A	Please perform the leftover from the previous lab sheet which is the **Task02** writing the Literature Review on Facial Expression Recognition (FER).	55	In this task, please work on your literature review section of your assignment which is the review of Facial Expression Recognition methods using deep learning. Completing this task will help you build understanding how FER works and a step towards finishing one section of your assignment.
	Break	05	
B	Please perform the **Task01** given on Page 2 for applying Transfer Learning approach.	55	This task is the implementation of Facial expression recognition system using Pretrained **Resnet18** and Transfer learning approach. You can start from Page **2** to Page **7**. First you will train the model and then you will test the model.

For One Hour Lab Session:

#	Task	Time (m)	Guidance
1	Please perform the Task **B** from above two hours lab session with the given instructions	50	Follow the related guide in Task **B**.

I recommend downloading this PDF file so that you can view each line of code clearly.

Applying Convolutional Neural Network and its Implementation for Facial Expression Recognition

Important Note: Every student advised to carefully review each line and develop a clear understanding of the concepts. As we progress to more advanced and technical sessions in the coming weeks, it is essential to have a strong grasp of these foundational concepts. This will also support your preparation for the second assessment.

Task01: Facial Expression Recognition using pretrained ResNet18, applying Transfer Learning

Last week, we explored and worked on the transfer learning TL considering VGG16 and found that the model got significant performance, and it was easy to update model parameters, etc. Also, we used Tensorflow and Keras for importing the models/networks which we setup for training. This week, we will apply TL using ResNet18 which is a deep CNN as part of the Residual Networks (ResNet) family, which won the ILSVRC 2015 competition. It consists of 18 layers, including convolutional layers, batch normalization, ReLU activations, and fully connected layers. The standout feature of ResNet18 is its use of residual blocks, which introduce skip connections that bypass one or more layers. This architecture addresses the vanishing gradient problem, allowing for deeper networks without performance degradation. ResNet18 is widely used for image classification, feature extraction, and transfer learning, offering a good balance between model complexity and performance, especially on smaller datasets or with limited computational resources.

About Pytorch and why we are using it in this sheet? Apart from this, as the ResNet18 isn't available in Tensorflow/Keras libraries, we will be using pytorch which is open-source deep learning framework developed by Facebook's AI Research lab (FAIR), used for building and training neural networks, offering a dynamic computational graph (eager execution) that makes model development and debugging more intuitive and Pythonic. You will see that PyTorch provides powerful tools like torch.nn for building neural networks, torch.optim for optimization algorithms, and torch.utils.data for data handling and preprocessing. Its GPU acceleration through CUDA, along with support for automatic differentiation using torch.autograd, makes it a preferred choice for researchers and developers alike. PyTorch's flexibility, coupled with a strong community and ecosystem, supports a wide range of applications, from computer vision and natural language processing to reinforcement learning and generative models. First import the essential libraries.

```
import cv2
import numpy as np
import torch
import torch.nn as nn
import torchvision.transforms as transforms
from torchvision import models
from torch.utils.data import DataLoader
from torchvision.datasets import ImageFolder
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import os
```

This code is specific to Google Colab and is used to access files stored in Google Drive.

```
from google.colab import drive
drive.mount('/content/drive')
```

We already used this cascade for face detection in previous semester which is very common in detecting the faces in an image. We use it again to detect the faces before processing. In the below snippet, lets load it.

```
# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier("/content/drive/My Drive/haarcascade_frontalface_default.xml")
```

Next, the CascadeClassifier in OpenCV loads a pre-trained Haar Cascade model to detect faces in images. The detect_face function first converts the image to grayscale using cv2.COLOR_BGR2GRAY, which simplifies the data and improves detection speed and accuracy. The detectMultiScale method is then applied to detect faces in the image, where scaleFactor=1.1 defines the image size reduction at each scale, minNeighbors=5 adjusts the quality of detected faces, and minSize=(30, 30) sets the minimum size for detected faces. If no face is detected, the function returns None to handle the absence of faces gracefully. When a face is found, the function crops the first detected face region and resizes it to 224x224 pixels, ensuring compatibility with the ResNet18 model, which requires a specific input size.

```
[4]
def detect_face(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    if len(faces) == 0:
        return None  # No face detected

    x, y, w, h = faces[0]
    face = image[y:y+h, x:x+w]
    face = cv2.resize(face, (224, 224))
    return face
```

Next is to load the train and test folder. The paths are added. Further, the image transformations is to make the image able to be processed efficiently. It applied include Resize, ToTensor, and Normalize operations. The Resize transformation scales images to 224x224 pixels, ensuring compatibility with the ResNet18 model, which requires this specific input size. The ToTensor step converts images to PyTorch tensors, enabling them to be used efficiently in the deep learning pipeline. Finally, the Normalize transformation standardizes the images using the mean and standard deviation values of the ImageNet dataset, aligning the input data with the pre-trained ResNet18 model's expectations and helping to stabilize and speed up model training.

```
[5]
# Define dataset paths
train_dir = '/content/drive/My Drive/CK_dataset/CK_dataset/train'
test_dir = '/content/drive/My Drive/CK_dataset/CK_dataset/test'

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

train_dataset = ImageFolder(train_dir, transform=transform)
test_dataset = ImageFolder(test_dir, transform=transform)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
```

ResNet18:

- **Pre-trained on ImageNet** (pretrained=True).
- Efficient for transfer learning.

Adjusting Final Layer:

- Replaces the fully connected layer to match the number of output classes (**6 classes**).

The **CrossEntropyLoss** function is well-suited for multi-class classification tasks, as it combines the LogSoftmax and Negative Log-Likelihood (NLLLoss) in a single, efficient step. This loss function penalizes the model more when its predicted class probabilities deviate from the true labels, promoting accurate predictions. The Adam optimizer is used with a learning rate of 0.0001, offering a good balance between speed and stability during training. Adam (Adaptive Moment Estimation) combines the advantages of both AdaGrad and RMSProp, adjusting the learning rate for each parameter dynamically, which helps in efficient convergence and handling sparse gradients.

Then we use:

GPU Acceleration (cuda): Uses GPU if available, otherwise defaults to CPU (cpu).
Model to Device: Moves the model to the specified device.

```
[6]
# Load ResNet18 model
model = models.resnet18(pretrained=True)
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, 6)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
```

[32]

```python
# Training with Test Accuracy per Epoch
num_epochs = 10
train_acc_list = []
test_acc_list = []
train_loss_list = []
test_loss_list = []

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    train_accuracy = 100 * correct / total
    train_acc_list.append(train_accuracy)
    train_loss_list.append(running_loss / len(train_loader))

    # Evaluate on Test Set after each epoch
    model.eval()
    correct_test = 0
    total_test = 0
    test_loss = 0.0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            test_loss += loss.item()
            _, predicted = outputs.max(1)
            total_test += labels.size(0)
            correct_test += predicted.eq(labels).sum().item()
    test_accuracy = 100 * correct_test / total_test
    test_acc_list.append(test_accuracy)
    test_loss_list.append(test_loss / len(test_loader))
    print(
    f'Epoch [{epoch+1}/{num_epochs}], Loss: {running_loss/len(train_loader):.4f}, '
    f'Train Accuracy: {train_accuracy:.2f}%, Test Accuracy: {test_accuracy:.2f}%'
)
```

During the training phase, the model is set to training mode using **model.train()**, enabling features like dropout and batch normalization to function appropriately. Several metrics are initialized, including **running_loss** to accumulate the total loss, and correct and total to compute training accuracy. Input images and labels are moved to the appropriate device (CPU or GPU) using .to(device) for faster processing. The gradient descent process involves several key steps: zeroing the gradients **(optimizer.zero_grad())** to prevent accumulation, performing a forward pass (outputs = model(images)) to generate predictions, calculating the loss (loss = criterion(outputs, labels)) using the specified loss function, executing a backward pass **(loss.backward())** to compute gradients, and updating model weights using **optimizer.step().** After each epoch, the model enters the testing phase by switching to evaluation mode **(model.eval())**, which disables dropout and normalizes batch statistics. During evaluation, gradient calculations are turned off using **torch.no_grad()**, which reduces memory usage and speeds up inference.

```python
# Final Evaluation and Confusion Matrix
model.eval()
y_true = []
y_pred = []
total = 0
correct = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        _, predicted = outputs.max(1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

final_test_accuracy = 100 * correct / total
print(f'Final Test Accuracy: {final_test_accuracy:.2f}%')

conf_matrix = confusion_matrix(y_true, y_pred)
cm_display = ConfusionMatrixDisplay(conf_matrix, display_labels=test_dataset.classes)

plt.figure(figsize=(8, 6))
cm_display.plot(cmap=plt.cm.Blues)
plt.show()
```

If you want to plot the training and testing plots, add the below code.

```
[34]  # Plot Accuracy and Loss
      plt.figure(figsize=(12, 5))

      # Accuracy Plot
      plt.subplot(1, 2, 1)
      plt.plot(train_acc_list, label='Train Accuracy')
      plt.plot(test_acc_list, label='Test Accuracy')
      plt.xlabel('Epochs')
      plt.ylabel('Accuracy')
      plt.legend()
      plt.title('Accuracy per Epoch')

      # Loss Plot
      plt.subplot(1, 2, 2)
      plt.plot(train_loss_list, label='Train Loss')
      plt.plot(test_loss_list, label='Test Loss')
      plt.xlabel('Epochs')
      plt.ylabel('Loss')
      plt.legend()
      plt.title('Loss per Epoch')

      plt.tight_layout()
      plt.show()
```

Part of Task01: Single Image Test.

So, now we will load / deploy the trained model to test a single image. We will use the same kind of pre-processing, or any steps applied on the image in training stage, to the testing one. For testing, visit the below link and copy/paste the code in the next cell of the above code.

https://colab.research.google.com/drive/1R_cWUqgg16bJ0HUewO6ZzWWpq48RP1iG?usp=sharing

Note: This model might overfit, and you need to exercise how to avoid overfitting (adding dropout layer, batch normalization layer, etc.)